Week 5 Assignment 2 Discussion

From Problem to Requirements

OVERVIEW

Complete SDLC Roadmap for Assignment 2- Case Study

1. Problem Understanding & Requirements Engineering
2. Software Modelling & Architecture/Design
3. Python Implementation (OOP Concepts)
4. Testing, Verification, Debugging & Refactoring
5. Version Control, Collaboration, Deployment & Maintenance
6. Professional Responsibility in Software Engineering

Problem Understanding & Requirements Engineering

Problem Understanding

- Identify stakeholder pain points and core needs.
- Define clear project scope boundaries.
- Differentiate between business goals and technical tasks.
- Avoid 'scope creep' early by documenting assumptions.

Requirements Engineering

- Functional: What the system must do (e.g., user login).
- Non-Functional: How it performs (e.g., latency < 200ms).
- Elicitation: Interviews, workshops, and user stories.
- Validation: Use cases and formal requirement sign-offs.

Software Modelling, Architecture & Design

Software Modelling

- Visualising systems using Unified Modelling Language (UML).
- Class Diagrams: Map structural classes and relationships.
- Use Case Diagrams: Define external actor system access.
- Sequence Diagrams: Chart runtime object interactions over time.

Architecture & Design Patterns

- Architectural Styles: MVC, Microservices, Layered.
- Design Principles: SOLID (Single Responsibility, etc.).
- Creational Patterns: Singleton, Factory Method.
- Structural & Behavioural: Adapter, Observer pattern rules.

Python OOP Implementation Foundations

Core Paradigm Realisation

- Classes & Objects: Define blueprints and runtime instances.
- Encapsulation: Restricting data access via private attributes (`__var`).
- Inheritance: Extending base classes to promote reuse.
- Polymorphism: Overriding methods dynamically via duck typing.

Pythonic Implementation Tips

- Leverage properties (`@property`) for clean getters/setters.
- Utilise composition over inheritance for high flexibility.
- Abstract Base Classes (`abc` module) guarantee interfaces.
- Type hinting (`typing`) enhances linting and data clarity.

Testing, Verification, Debugging & Refactoring

Testing & Verification

• Unit Testing: Isolate and evaluate single functions/classes via `unittest` or `pytest`.

• Integration Testing: Ensure components interface correctly.

• Test-Driven Development (TDD): Red, Green, Refactor cycle.

• Coverage: Gauge percentage of lines evaluated by tests.

Debugging & Refactoring

• Debugging: Use `pdb` breakpoints instead of basic print statements.

• Refactoring: Enhance internal structure without altering behavior.

• Code Smells: Remediate long methods, duplication, and God classes.

• Static Analysis: Enforce style rules via `flake8`, `black`, and `mypy`.

Collaboration, Deployment & Maintenance

Version Control & Collaboration

- Git Fundamentals: Commits, branching strategies, and merges.
- Feature Branch Workflow: Isolate active development safety.
- Pull Requests (PRs): Peer reviews and mandatory approvals.
- Conflict Resolution: Mitigating overlapping file modifications.

Deployment & Maintenance

- CI/CD Pipelines: Automate tests and staging distributions.
- Containerisation: Package environments uniformly using Docker.
- Monitoring: Track application uptime and execution errors.
- Refactoring: Patch issues, update libraries, and manage technical debt.

Professional Responsibility in Software Engineering

Ethical Software Development: Prioritising user privacy, data security, and systemic transparency. Adhering to the ACM/IEEE Software Engineering Code of Ethics.

Code Sustainability: Writing self-documenting code with comprehensive docstrings and comments. Leaving codebases cleaner than you initially found them.

Inclusivity & Accessibility: Ensuring systems accommodate users of all abilities and diverse backgrounds.

Accountability: Owning your system defects, rigorously reporting security risks, and refusing intentionally deceptive design architectures.

Week 5 Assignment 2- Stage 2

- From Problems to Requirements
  - Central question: How do we know what software should actually do?
  - Week 5 | Software Engineering with Python and GenAI
- Problem -> Stakeholders -> Scope -> Requirements -> User Stories -> Acceptance Criteria
  - AI role: requirements-analysis assistant
  - SmartCare: Requirements Specification v1.0

Why Requirements Matter ?

- Correct code can still solve the wrong problem
- Requirements connect stakeholder needs to engineering work
- Good requirements reduce rework
- The cost of building wrong thing !
  - A perfectly built feature has little value if nobody needs it
  - AI can accelerate the wrong solution as easily as the right one

How is requirements engineering done ?

- Elicit – Analyse – Document – Validate - Manage change
- Elicit Requirements
  - Who uses the system?
  - Who is affected?
  - Who manages or pays for it?
  - Who supports it?
- Is this Stakeholder ?

Who Is a Stakeholder?

- A person, group or organisation with an interest in the system
- Stakeholders may be direct users or indirectly affected
- SmartCare Stakeholders ?
  - Receptionist
  - Practitioner
  - Clinic manager
  - Patient
  - System administrator

Stakeholder Needs Can Differ

- Receptionist: efficient booking
- Practitioner: accurate schedule
- Manager: reliable records
- Patient: correct appointment information
- From Stakeholder needs to Requirements
  - Stakeholder need -> clarified behaviour -> requirement
  - Do not jump directly from vague need to implementation

What is a requirement ?

- A documented statement of required behaviour, quality or constraint
- Clear enough to guide design and verification
- Functional Requirements
- Non Functional Requirments

Functional Requirements (FR)

- Functional Requirements (FR) - What the system must do ?
  - Register patient
  - Book/cancel appointment
  - Search patient
- SmartCare Functional Requirement Examples
  - FR-01 Register patient
  - FR-02 View patient details
  - FR-03 Create appointment
  - FR-04 Cancel appointment
  - FR-05 Retain history

Non-Functional Requirements (NFR)

- Qualities or constraints, Usability, Reliability, Maintainability, Performance, Data integrity
- SmartCare NFR Examples
  - NFR-01 Invalid operations must not corrupt data
  - NFR-02 Business logic should be independently testable
  - NFR-03 Common operations should respond without noticeable delay

Functional vs Non-Functional

- Functional = what the system does
- Non-functional = how well or under what constraints
- Both affect design and testing
- Requirements must be testable
  - Clear
  - Specific
  - Feasible
  - Consistent
  - Traceable and
  - Verifiable

Poor and Ambiguous Requirements

- Poor Requirement: Easy to Use
  - The system should be easy to use.'
    - Easy for whom? For which task? How could we verify it?
- Improve Ambiguous Requirements
  - Replace vague adjectives with observable criteria
  - Define user, task and outcome
  - Do not invent false precision
  - Ambiguity in Natural Language
    - fast, simple, secure, appropriate, usually, quickly
    - Different readers may interpret them differently
  - Common Ambiguity Traps
    - Unclear pronouns
    - Missing conditions
    - Undefined terms
    - Several requirements in one sentence
    - Hidden assumptions

Requirements Need Evidence

- Requirement = stakeholder evidence or explicit assumption
- AI suggestion is not evidence
- Sources of Evidence
  - Client brief
  - Stakeholder interview
  - Business rule
  - Existing process
  - Policy
  - Explicit assumption
- Confirmed – Assumed- Unsupported Requirements
  - Confirmed – has evidence from valid sources
  - Assumption - requires validation
  - Unsupported – No evidence
  - Out of scope ??

Scope Matters

- Scope defines what the project will and will not deliver
- Scope protects time and quality
- SmartCare Scope
  - Patient registration/search
  - Practitioner management
  - Appointments
  - History
  - Persistence
  - Basic validation
- SmartCare Out of Scope
  - Diagnosis
  - Prescriptions
  - Billing
  - Insurance integration
  - Real patient portal
  - Clinical decision support

Scope Creep, Constraints and Assumptions

- Scope Creep
  - Unapproved features increase complexity
  - AI often proposes attractive extras
  - More features do not automatically mean more value
- Constraints
  - Technology, Time, Data, Policy, Course constraints
- Assumptions
  - Fills a gap temporarily
  - Must be visible
  - Important assumptions should later be confirmed or rejected

Requirements Traceability and User Stories

- Problem -> Requirement -> Design -> Code -> Test
- Traceability explains why an artefact exists
- Numbering Requirements
  - FR-01, FR-02 ...
  - NFR-01, NFR-02 ...
  - Stable identifiers support discussion and testing
- User Stories
  - As a <role>, I want <capability>, So that <benefit>
- Anatomy of a User Story
  - Role: who?, Capability: what?, Benefit: why?
- SmartCare User Story Example
  - As a receptionist, I want to cancel an appointment so that the clinic schedule remains accurate
- But User Stories are not enough
  - User stories are concise
  - Detailed behaviour still needs acceptance criteria and business rules

Requirements Acceptance Criteria

- Conditions that determine whether a story is:
  - Satisfied, Observable and Testable
- Given - When – Then
  - GIVEN a starting context
  - WHEN an action occurs
  - THEN an observable outcome follows
- SmartCare Booking Acceptance Criteria
  - GIVEN a registered patient and available practitioner
  - WHEN the receptionist books
  - THEN store with status SCHEDULED
- Negative Acceptance Criteria
  - GIVEN practitioner already booked at 10:00
  - WHEN another active booking is requested
  - THEN reject with meaningful conflict error

Requirement, Testing and AI

  - Requirement -> Acceptance criterion -> Test scenario -> pytest test
  - Requirements become verification evidence later
- AI as Requirements Assistant
  - Identify ambiguity
  - Generate clarification questions
  - Critique wording
  - Do not invent confirmed requirements
- What AI does well
  - Alternative wording, Possible ambiguity, Clarification questions, Edge cases and Organisation
- What AI can get wrong
  - Invent stakeholders, Invent business rules, Add fashionable features, Over-specify without evidence, and Present assumptions as facts
- Try this prompt to learn how AI Expands a Vague Requirement
  - 'SmartCare needs a modern appointment system that is easy to use.'
  - Ask AI for requirements and observe what it invents
- Classify AI Suggestions
  - Confirmed,
  - Assumption requiring validation
  - Unsupported
  - Out of scope

Engineering Prompt for Requirements Review

- Context: SmartCare brief
  - Task: identify ambiguities/questions
  - Constraint: do not invent requirements
  - Output: evidence / uncertainty / question
- SmartCare v0.2 Deliverable
  - Stakeholder analysis
  - Scope
  - 8-12 FRs
  - 4-6 NFRs
  - 4-6 user stories
  - Given-When-Then criteria
  - AI review record
- Stage 2 Rules
  - Engineering Rule: Requirements need evidence
  - AI Rule: AI can discover questions; it cannot decide stakeholder requirements
- Next Week – Stage 3: From Requirements to Software Models